

Sistemas Operacionais

- Programa x Processo
- Concorrência x Paralelismo
- Áreas de memória de um processo
- Troca de contexto
- Ciclo de vida dos processos
- Processos no ambiente UNIX
- Threads
- Fork



Windows



Mac OS



Linux



Chrome OS



Android



iOS



Unix



BSD

Programa

- **Programa:** conjunto de instruções e dados usados para realizar uma tarefa.
- **Programa fonte:** é o código escrito pelo programador, geralmente em linguagens de alto nível (como C, Java, etc.) ou, às vezes, em linguagem de baixo nível (assembler).
- **Compilado x Interpretado:**
 - **Compilado:** o código fonte é transformado em linguagem de máquina antes de executar.
 - **Interpretado:** o programa precisa de um interpretador para ser executado.
- **Programa executável:** é o resultado final, já em linguagem de máquina, pronto para ser executado pelo computador.

Programa

Código Fonte

- Armazenado em arquivo
- Contém uma sequência de dados e comandos em linguagem de alto nível representados como uma sequência de caracteres ASCII.

Compilador

Programa Executável

- Armazenado em arquivo
- Contém uma seqüência de instruções e dados codificados em linguagem de máquina

Estrutura de Programa Executável

- **Cabeçalho**
 - Informações sobre as demais áreas
 - Qual é o formato do executável; arquitetura utilizada (x86, x86-64, ARM etc.); endereço de entrada do programa; quais regiões precisam ser carregadas na memória; permissões de cada região.
- **Área de código:**
 - Contém as instruções em linguagem de máquina. Não é modificada durante a execução.
- **Área de dados**
 - Dados iniciados
 - Contém as variáveis e estruturas globais que possuem
 - valores definidos inicialmente
 - Constantes
 - Dados não iniciados
 - Variáveis e estruturas não inicializadas
- **Tabela de símbolos**
 - Tabela que contém o símbolo (nome de função, nome de variável, nome de estrutura, nome de *label*, nome de constante, etc) e o endereço de memória virtual de cada símbolo.



Estrutura da Memória de um Programa

Stack: cresce e diminui automaticamente conforme as funções são chamadas e encerradas.

Heap: permite **alocação dinâmica**, controlada pelo programa.

BSS: globais (estáticas) não inicializadas

Data : globais (estáticas) inicializadas

Text: código executável.

Estrutura de Programa Executável

No Linux, faça o programa:

```
#include <stdio.h>
int variavel01;
int variavel02=1000;
int main()
{
    char nome[50];
    printf("\nInforme seu Nome:\n");
    scanf("%s",nome );
    printf ("\n \n  Ola %s \n ", nome);
    return 0;
}
```

Estrutura de Programa Executável

No Linux, faça o programa:

Utilize o utilitário “nm” (nm <binario>) e relacione as informações relativas às

- **Áreas do executável**
 - Área de código (text area)
 - Área de dados
 - Área da pilha de execução
- **Símbolos do programa e seus endereços de memória virtual**
 - Sub-rotinas e dados

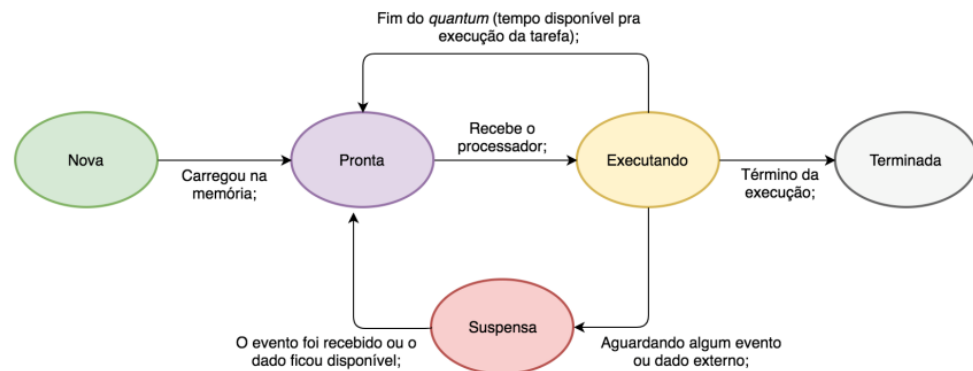
Dica:

- **A** – Símbolo com endereço absoluto
- **B** – Dados não iniciados (BSS area)
- **D** – Dados iniciados (Data area)
- **R** – Dados read only
- **T** – Text (código)
- **Minúscula**: variável local

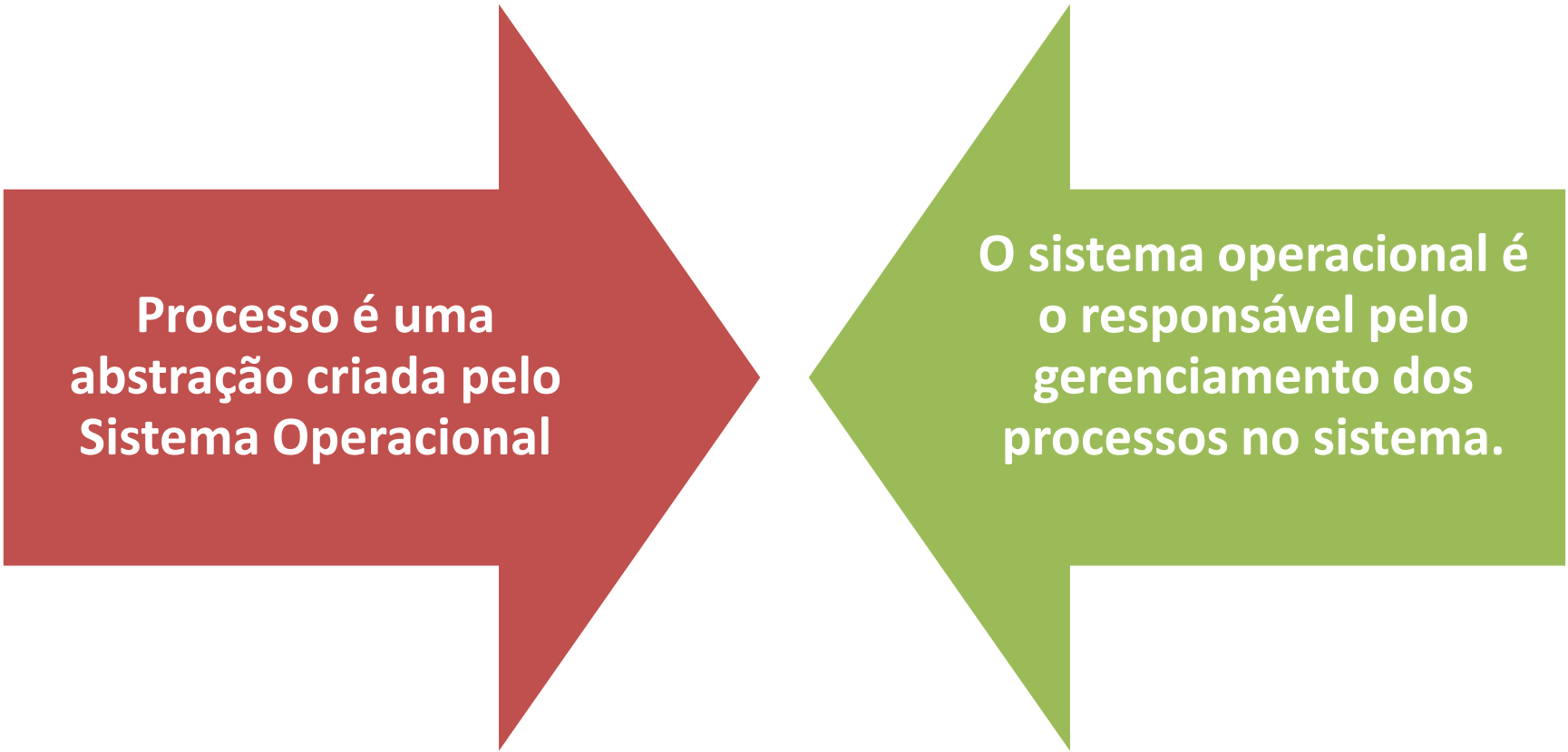
Dica: use o utilitário “size” para verificar o tamanho das áreas do binário.

Processo

- Um programa sendo executado
- Possui um contexto (informações) como:
 - Informações de controle
 - Geralmente armazenadas na tabela de processos
 - Informações:
 - identificação única (pid - process identification)
 - Registradores
 - Estado do processo
 - Identificação do usuário
 - Terminal do qual foi disparado
 - Áreas de memória
 - Área de código
 - Área de dados
 - Área da pilha de execução
 - Outras áreas de memória



Processo



**Processo é uma
abstração criada pelo
Sistema Operacional**

**O sistema operacional é
o responsável pelo
gerenciamento dos
processos no sistema.**

Processo

Sistemas de computação modernos criam a ilusão de que vários processos (aplicações) executam ao mesmo tempo no sistema.



Na realidade, em sistemas monoprocessadores, em um determinado instante existe somente um programa sendo executado pela CPU.



O sistema operacional gerencia o uso da CPU de forma que seja executado “um pouco” de cada processo por vez. Este chaveamento entre os processos é tão rápido que cria a ilusão de que os processos executam simultaneamente.

Concorrência x Paralelismo

Concorrência

- Pseudo paralelismo
- Ilusão de execução simultânea de processos devido ao rápido chaveamento entre suas execuções em uma única CPU

Paralelismo

- Paralelismo real
- Em sistemas com mais que uma CPU cada uma pode estar executando um processo efetivamente em paralelo.

Contexto de um processo

É o conjunto de informações relevantes à execução do processo:

- **Contexto de Hardware:**
 - Registradores da CPU
- **Contexto de Software:**
 - Área de código
 - Área de dados
 - Área da pilha de execução
 - Identificação do processo (pid – process identification)
 - Estado
 - Prioridade
 - Conjunto de arquivos abertos

Exemplo: Comando “ps”



Troca de Contexto

Atividade de mudança de contexto de processos em um ambiente de computação.

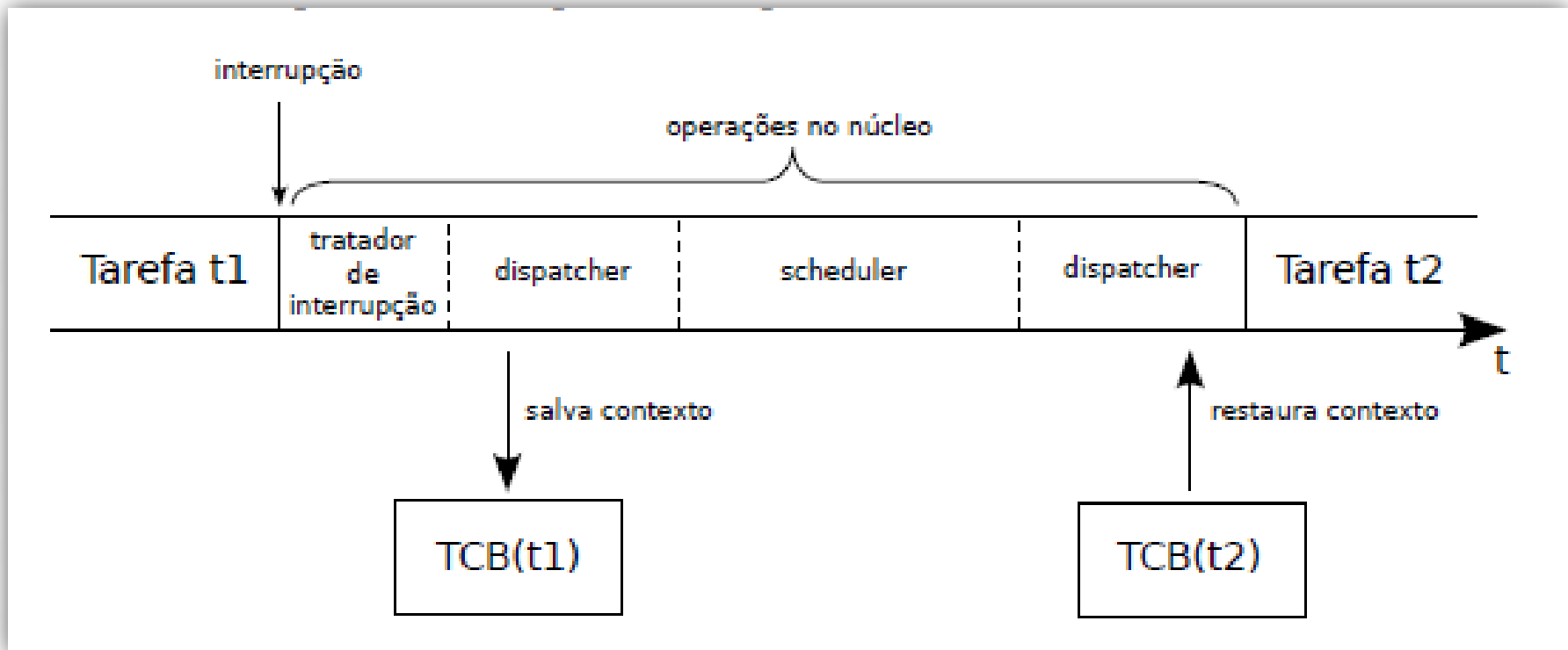
- **Atividade realizada pelo Sistema Operacional**
- **Envolve a troca de contexto de hardware e software de um processo:**
 - Salvamento do contexto do processo corrente
 - Retomada do contexto do outro processo

Troca de Contexto

- **Atividade de mudança de contexto de processos em um ambiente de computação**
- **Atividade realizada pelo Sistema Operacional**
- **Envolve a troca de contexto de hardware e software de um processo:**
 - Salvamento do contexto do processo corrente
 - Retomada do contexto do outro processo

TROCA DE CONTEXTO

Necessária quando interrompe a execução de uma tarefa para retornar a ela mais tarde, sem corromper seu estado interno.



TAREFA

Uma tarefa é definida como sendo a execução de um fluxo sequencial de instruções, construído para atender uma finalidade específica.

PROGRAMA $\neq$ TAREFA



TAREFA

- Um programa é um conjunto de uma ou mais sequencias de instruções escritas para resolver um problema específico, constituindo assim uma aplicação ou utilitário.
- Uma tarefa é a execução, pelo processador, das sequencias de instruções definidas em um programa para realizar seu objetivo.

TIPOS DE TAREFAS: THREADS, PROCESSOS

TAREFA

- **O PROGRAMA:** representa um **conceito estático**, sem um estado interno definido (que represente uma situação específica da execução) e sem interações com outras entidades (o usuário ou outros programas)
- **A TAREFA:** Trata-se de um **conceito dinâmico**, que **possui um estado interno** bem definido a cada instante (os valores das variáveis internas e a posição atual da execução) e interage com outras entidades: o usuário, os periféricos e/ou outras tarefas.

TAREFA

Analogia com Receita de Bolo:

- **O programa** é o equivalente de uma “receita de bolo” dentro de um livro de receitas (um diretório) guardado em uma estante (um disco) na cozinha (o computador).
- **“Executar”** a receita, providenciando os ingredientes e seguindo os passos definidos na receita, é a tarefa propriamente dita.
- A cada momento, a cozinheira (o processador) está seguindo um passo da receita (posição da execução) e tem uma certa disposição dos ingredientes e utensílios em uso (as variáveis internas da tarefa).





O GERENCIAMENTO DA TAREFA

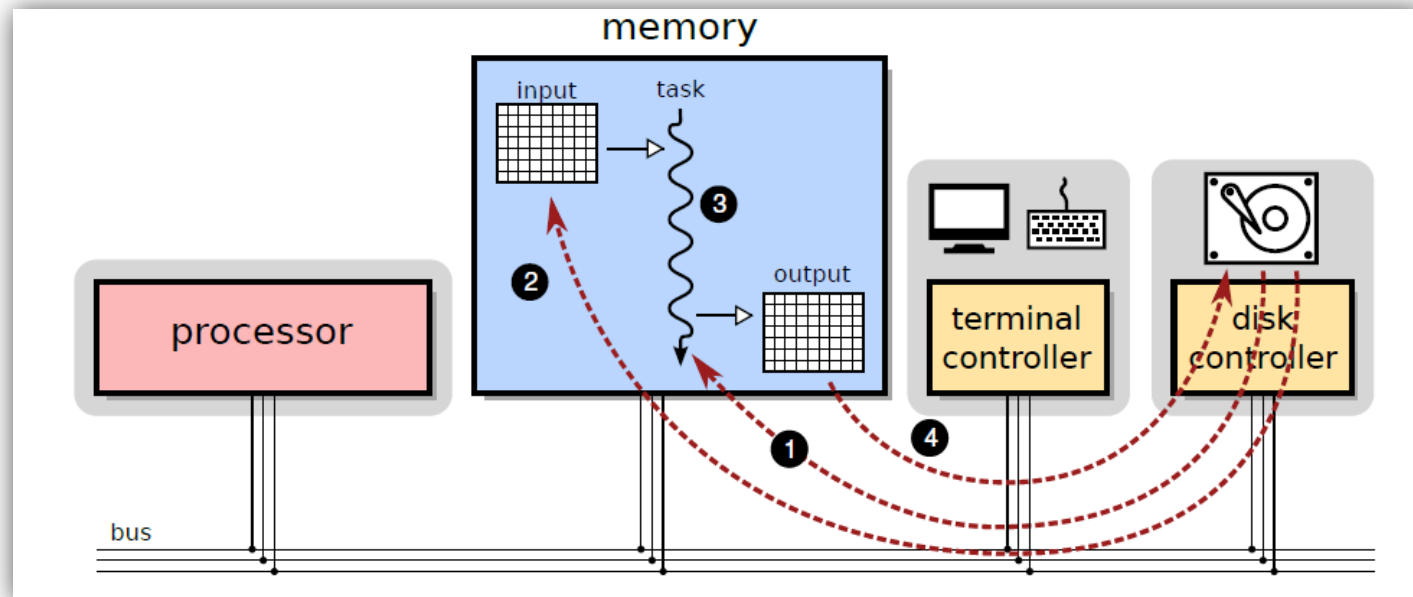
- Em um computador, o processador tem de **executar todas as tarefas** submetidas pelos usuários.
- Essas tarefas geralmente têm **comportamento, duração e importância** distintas.
- Cabe ao sistema operacional organizar as tarefas para executá-las e decidir em que ordem fazê-lo.

A GERENCIA DA TAREFA

- **Sistemas monotarefa**
 - Cada programa binário era carregado do disco para a memória e executado até sua conclusão.
- **Sistemas multitarefas**
 - Várias tarefas podiam estar em andamento simultaneamente: uma estava ativa (executando) e as demais prontas (esperando pelo processador) ou suspensas (esperando dados ou eventos externos).
- **Sistemas de tempo compartilhado**
 - Para cada tarefa que recebe o processador é definido um prazo de processamento, denominado fatia de tempo ou “*quantum*” (uma pequena porção).

SISTEMAS MONOTAREFA

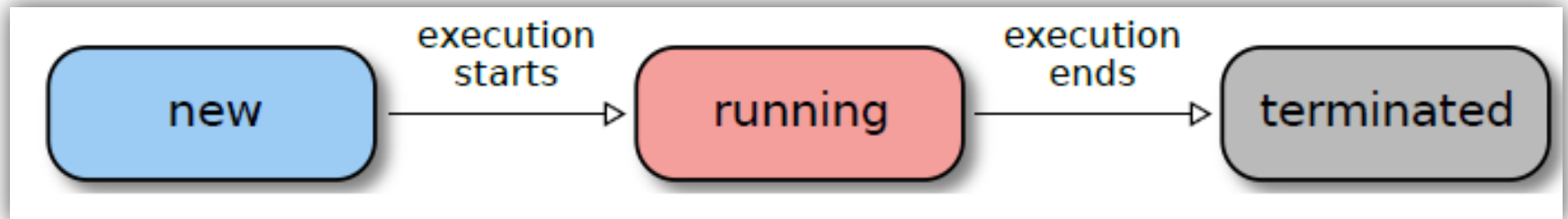
- Primeiros sistemas de computação (1940)
- Uma (01) tarefa por vez
 - Sem monitor (Controle Humano)
 - Com monitor (FILA DE PROGRAMAS)



SISTEMA MONOTAREFA

ESTADOS DAS TAREFAS

Nesse método de processamento de tarefas é possível delinear um diagrama de estados para cada tarefa executada pelo sistema:



SISTEMA MONOTAREFA

Cada Tarefa era carregado pelo Operador.



O MONITOR DE SISTEMA

Com a evolução do hardware, as tarefas de carga e descarga de código entre memória e disco, coordenadas por um operador humano, passaram a se tornar críticas.

Mais tempo era perdido nesses procedimentos manuais que no processamento da tarefa em si.



Para resolver esse problema foi construído um **programa monitor**, que era carregado na memória no início da operação do sistema, com a função de gerenciar a execução dos demais programas.

O MONITOR DE SISTEMA

O programa monitor executava **continuamente** os seguintes passos sobre uma fila de programas a executar, armazenada no disco:

1. Carregar um programa do disco para a memória;
2. Carregar os dados de entrada do disco para a memória;
3. Transferir a execução para o programa recém carregado;
4. Aguardar o término da execução do programa;
5. Escrever os resultados gerados pelo programa no disco.

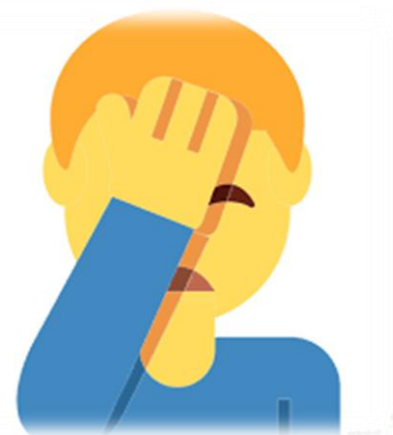
O MONITOR DE SISTEMA

Monitor agilizou o uso do processador.



Novo Problema:

Uma tarefa ficava aguardando recursos.



SISTEMAS MULTI-TAREFAS

Novo Problema:

- CPU muito mais rápida que dispositivos de E/S
- CPU ficava esperando a E/S

Solução:

SISTEMAS MULTI-TAREFAS

Processador suspende a execução da tarefa que espera dados externos e **passa a executar outra tarefa.**

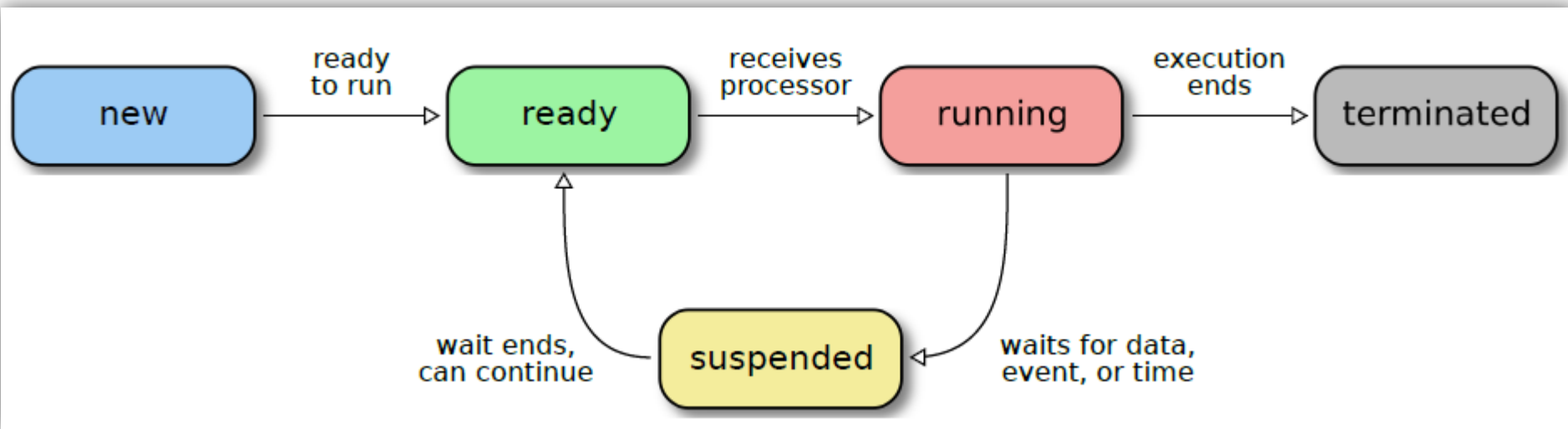


SISTEMAS MULTI-TAREFAS

- A solução encontrada para resolver esse problema foi permitir ao monitor suspender a execução da tarefa que espera dados externos e passar a executar outra tarefa.
- Mais tarde, quando os dados de que a tarefa suspensa necessita estiverem disponíveis, ela pode ser retomada no ponto onde parou. Para tal, é necessário ter mais memória (para poder carregar mais de um programa ao mesmo tempo) e criar mecanismos no monitor para suspender uma tarefa e retomá-la mais tarde.
- Uma forma simples de implementar a suspensão e retomada de tarefas de forma transparente consiste no monitor fornecer um conjunto de rotinas padronizadas de entrada/saída à tarefas; essas rotinas implementadas pelo monitor recebem as solicitações de entrada/saída de dados das tarefas e podem suspender uma execução quando for necessário, devolvendo o controle ao monitor.

SISTEMAS MULTI-TAREFAS

ESTADOS DAS TAREFAS



Várias tarefas podiam estar em andamento simultaneamente: uma estava ativa e as demais suspensas, esperando dados externos ou outras condições.

SISTEMAS MULTI-TAREFAS

Novo Problema:

E se uma tarefa não finalizasse? Se uma tarefa demorasse o resto da vida?





SISTEMAS DE TEMPO COMPARTILHADO

PROBLEMA DO LAÇO INFINITO: quando uma tarefa executar esse código, ela nunca encerrará nem será suspensa aguardando operações de entrada/saída de dados.

```
1 // calcula a soma dos primeiros 1000 inteiros
2
3 #include <stdio.h>
4
5 int main ()
6 {
7     int i = 0, soma = 0 ;
8
9     while (i <= 1000)
10         soma += i ;
11
12     printf ("A soma vale %d\n", soma);
13     exit(0) ;
14 }
```




SISTEMAS DE TEMPO COMPARTILHADO

SOLUÇÃO:



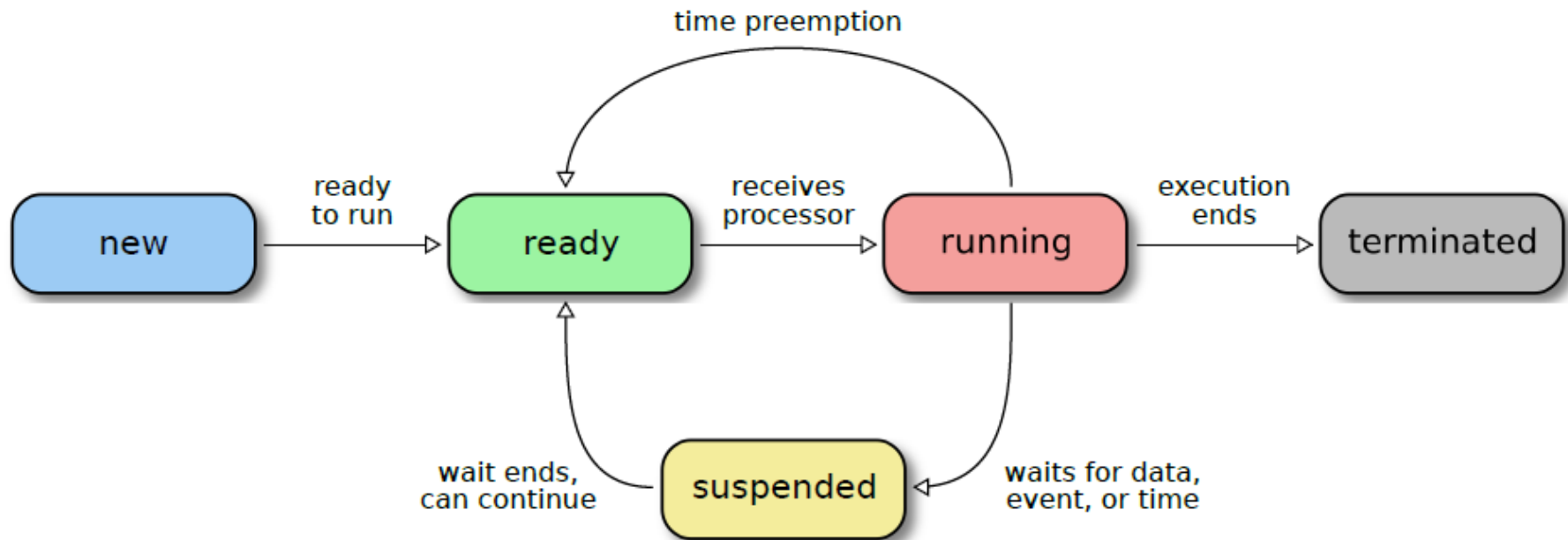
COMPARTILHAMENTO DE TEMPO

Para resolver essa questão, foi introduzido no início dos anos 60 um novo conceito: o compartilhamento de tempo, ou *time-sharing*, através do sistema **CTSS – Compatible Time-Sharing System**

SISTEMAS DE PREEMPTIVOS

- Nessa solução, para cada tarefa que recebe o processador é definido um prazo de processamento, denominado fatia de tempo ou quantum.
- Esgotado seu quantum, a tarefa em execução perde o processador e volta para uma fila de tarefas “prontas”, que estão na memória aguardando sua oportunidade de executar, e outra tarefa é ativada.
- O ato de retirar um recurso “**à força**” de uma tarefa (neste caso, o processador) **é denominado preempção**. Sistemas que implementam esse conceito são chamados **sistemas preemptivos**.
- Em um sistema operacional típico, a implementação da preempção por tempo usa as interrupções geradas por um *temporizador* programável disponível no hardware.

SISTEMAS DE PREEMPTIVOS

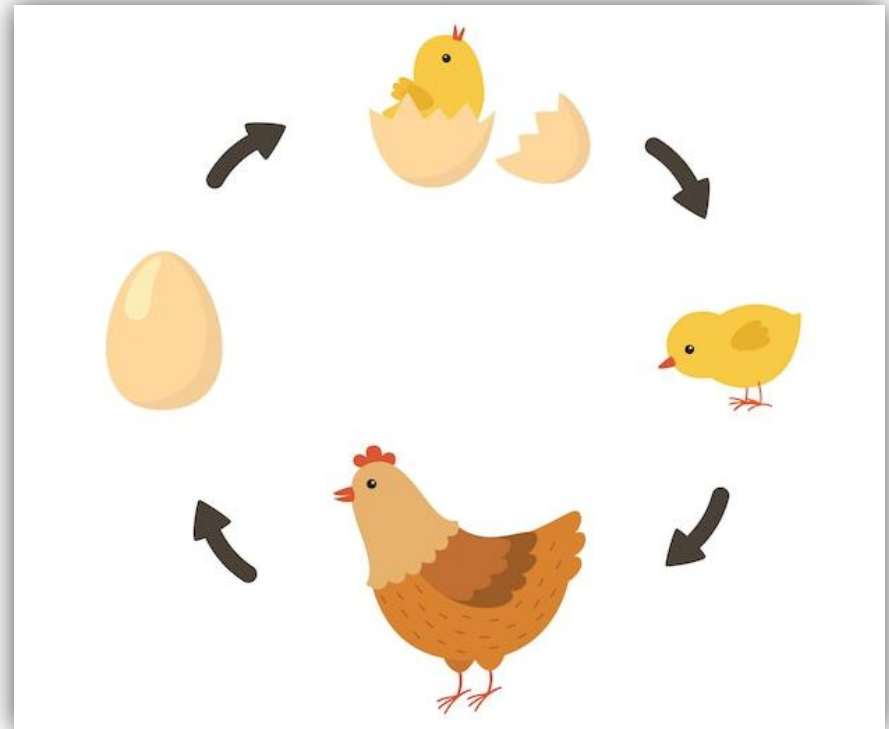


Cada tarefa recebe um quantum de tempo.
Quando o tempo terminar (Fim do quantum), perde o processador.

Implementado em hardware:

- Interrupção em tempo regulares
- Tratador de interrupção: move a tarefa para a lista de prontas.

Ciclo de vida das tarefas



Ciclo de vida das tarefas

Os estados e transições do ciclo de vida têm o seguinte significado:

- **Nova** : A tarefa está sendo criada.
 - Ex. eu código está sendo carregado em memória, junto com as bibliotecas necessárias, e as estruturas de dados do núcleo estão sendo atualizadas para permitir sua execução.
- **Pronta** : A tarefa está em memória, pronta para executar (ou para continuar sua execução), apenas aguardando a disponibilidade do processador.
 - Todas as tarefas prontas são organizadas em uma fila cuja ordem é determinada por algoritmos de escalonamento.
- **Executando** : O processador está dedicado à tarefa, executando suas instruções e fazendo avançar seu estado.
- **Suspensa** : A tarefa não pode executar porque depende de dados externos ainda não disponíveis (do disco ou da rede, por exemplo), aguarda algum tipo de sincronização (o fim de outra tarefa ou a liberação de algum recurso compartilhado) ou simplesmente espera o tempo passar (em uma operação sleeping, por exemplo).
- **Terminada** : O processamento da tarefa foi encerrado e ela pode ser removida da memória do sistema.

Ciclo de Vida das Tarefas

Tão importantes quanto os estados das tarefas apresentados são as transições entre esses estados:

- **Nova → Pronta** : ocorre quando a nova tarefa termina de ser carregada em memória, juntamente com suas bibliotecas e dados, estando pronta para executar.
- **Pronta → Executando** : esta transição ocorre quando a tarefa é escolhida pelo escalonador para ser executada, dentre as demais tarefas prontas.
- **Executando → Pronta** : esta transição ocorre quando se esgota a fatia de tempo destinada à tarefa (ou seja, o fim do quantum); como nesse momento a tarefa não precisa de outros recursos além do processador, ela volta à fila de tarefas prontas, para esperar novamente o processador.
- **Executando → Terminada** : ocorre quando a tarefa encerra sua execução ou é abortada em consequência de algum erro (acesso inválido à memória, instrução ilegal, divisão por zero, etc.). Na maioria dos sistemas a tarefa que deseja encerrar avisa o sistema operacional através de uma chamada de sistema (no Linux é usada a chamada `exit`).
- **Terminada →** : Uma tarefa terminada é removida da memória e seus registros e estruturas de controle no núcleo são apagadas.
- **Executando → Suspensa** : caso a tarefa em execução solicite acesso a um recurso não disponível, como dados externos ou alguma sincronização, ela abandona o processador e fica suspensa até o recurso ficar disponível.
- **Suspensa → Pronta** : quando o recurso solicitado pela tarefa se torna disponível, ela pode voltar a executar, portanto volta ao estado de pronta.



CONTEXTO: estado interno da tarefa

Cada tarefa possui um **TCB (Task Control Block)**

- Identificador da tarefa
- Estado da tarefa
- Informações de contexto do processador
 - (a posição de código (PC – program counter), os valores de suas variáveis, os arquivos, Stack Pointer (SP))
- Lista de áreas de memória usadas pela tarefa
- Recursos utilizados (Listas de arquivos abertos, conexões de rede, etc)
- Informações de gerência e contabilização (prioridade, usuário proprietário, data de início, tempo de processamento já decorrido, volume de dados lidos/escritos, etc.)

TROCA DE CONTEXTO

- **Dispatcher:** o armazenamento e recuperação do contexto e a atualização das informações contidas no TCB de cada tarefa são aspectos mecânicos, providos por um conjunto de rotinas denominado **despachante** ou **executivo**
- **Scheduler:** a escolha da próxima tarefa a receber o processador a cada troca de contexto é estratégica, podendo sofrer influências de diversos fatores, como as prioridades, os tempos de vida e os tempos de processamento restante de cada tarefa.
 - No Linux as operações de troca de contexto para a plataforma Intel x86 estão definidas através de diretivas em Assembly no arquivo `arch/i386/kernel/process.c` dos fontes do núcleo.

PROCESSOS



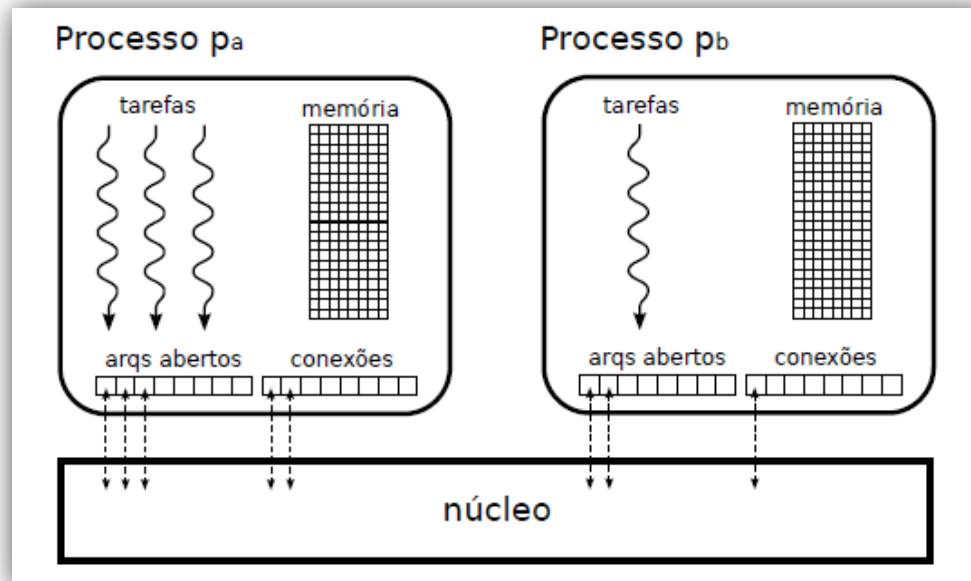
PROCESSOS

- **PROCESSOS são isolados entre si** pelos mecanismos de proteção providos pelo hardware (isolamento de áreas de memória, níveis de operação e chamadas de sistema) e pela própria gerência de tarefas, que atribui os recursos aos processos (e não às tarefas), impedindo que uma tarefa em execução no processo **A** acesse um recurso atribuído ao processo **B**.
- **TCB SIMPLIFICADO:** a troca de contexto entre tarefas vinculadas ao mesmo processo é muito mais simples e rápida que entre tarefas vinculadas a processos distintos, pois somente os registradores do processador precisam ser salvos/restaurados (as áreas de memória e demais recursos são comuns às duas tarefas).

PROCESSOS

Uma unidade de contexto contém um PCB (***Process Control Blocks***):

- PID (Process ID)
- Cada TAREFA tem um TCB simplificado:
 - Identificador, Registradores, ref. ao processo



CRIAÇÃO DE PROCESSOS

Eventos que criam processos:

- Início do sistema.
- Execução de uma chamada de sistema de criação de processo por um processo em execução.
- Uma requisição do usuário para criar um novo processo.
- Início de uma tarefa em lote (batch job).

Classificação:

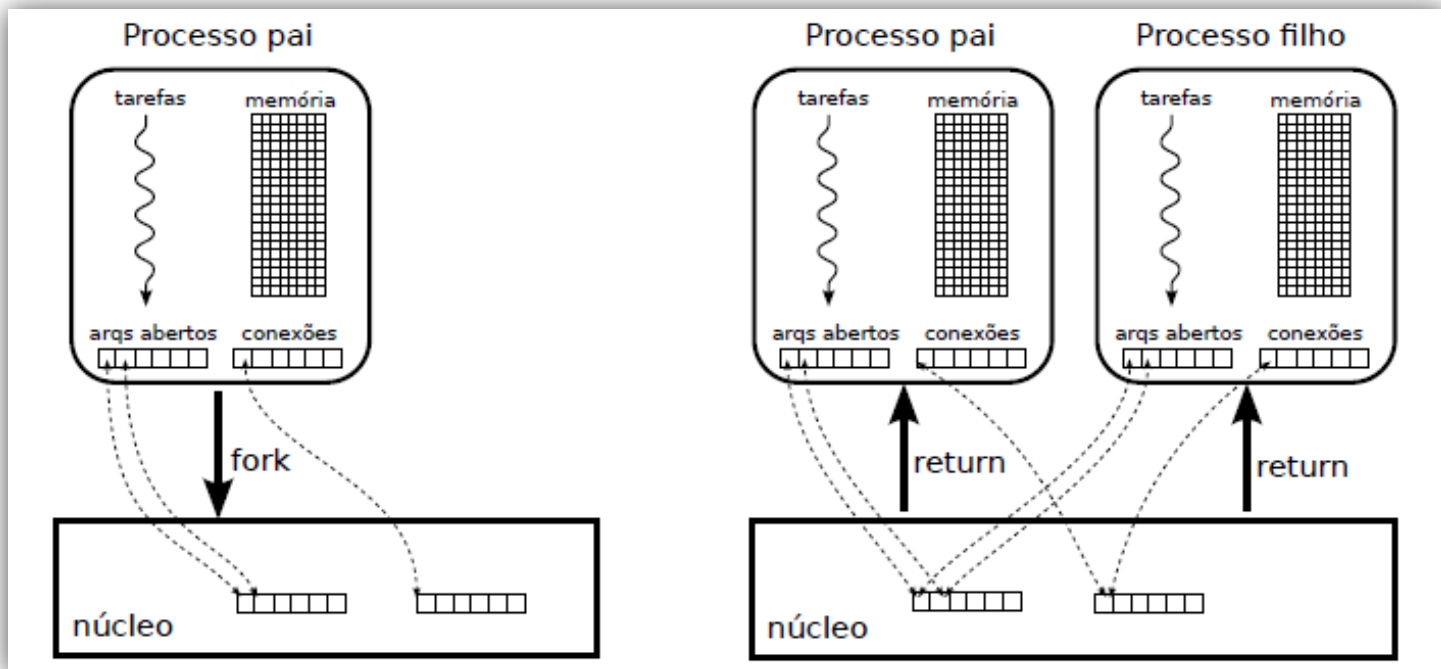
- **Foreground**: em primeiro plano
- **Background**: em segundo plano. Também chamados de **daemons**

CRIAÇÃO DE PROCESSOS

Feita por chamadas de sistemas.

- Exemplo no Unix:

fork → cria uma réplica do processo solicitante



CRIAÇÃO DE PROCESSOS

- **FORK:** todo o espaço de memória do processo é replicado, incluindo o código da(s) tarefa(s) associada(s) e os descritores dos arquivos e demais recursos associados ao mesmo.
- Ambos os processos têm os mesmos recursos associados, **embora em áreas de memória distintas.**

CRIAÇÃO DE PROCESSOS

```
1  #include <unistd.h>
2  #include <sys/types.h>
3  #include <sys/wait.h>
4  #include <stdio.h>
5  #include <stdlib.h>
6
7  int main (int argc, char *argv[], char *envp[])
8  {
9      int pid ;           /* identificador de processo */
10
11     pid = fork () ;      /* replicação do processo */
12
13     if ( pid < 0 )        /* fork não funcionou */
14     {
15         perror ("Erro: ") ;
16         exit (-1) ;      /* encerra o processo */
17     }
18     else if ( pid > 0 )   /* sou o processo pai */
19     {
20         wait (0) ;        /* vou esperar meu filho concluir */
21     }
22     else                  /* sou o processo filho*/
23     {
24         /* carrega outro código binário para executar */
25         execve ("/bin/date", argv, envp) ;
26         perror ("Erro: ") ; /* execve não funcionou */
27     }
28     printf ("Tchau !\n") ;
29     exit(0) ;             /* encerra o processo */
30 }
```

PROCESSOS Linux (FORK)

Um processo pode estar num dos seguintes estados:

- **idle/new:**

Processo recém criado (sendo preparado).

- **ready:**

Processo sendo carregado.

- **standby:**

O próximo a ser executado.

- **running:**

O processo esta rodando (ativo).

- **blocked:**

Esperando evento externo (entrada do usuário) (inativo).

- **suspended-blocked:**

Processo suspenso.

- **zombied:**

O processo acabou mas não informou seu pai.

- **done-terminated:**

O processo foi encerrado e os recursos liberados.

TÉRMINO DE PROCESSOS

Eventos que terminam processos:

- Saída normal (voluntária)
- Saída por erro (voluntária)
- Erro fatal (involuntário)
- Cancelamento por um outro processo (involuntário)

Comandos para Término de Processo:

- Unix: `exit`
- Windows: `ExitProcess`

CRIAÇÃO DE PROCESSOS

Processos sincronizados e não sincronizados:

- Quando o processo pai espera o encerramento do processo filho, ele tem uma execução sincronizada.
- Quando os processos rodam independentemente eles são não sincronizados.

A comunicação entre processos pode ser realizada utilizando-se variáveis de ambiente, pipes, ou memória compartilhada (shared memory).

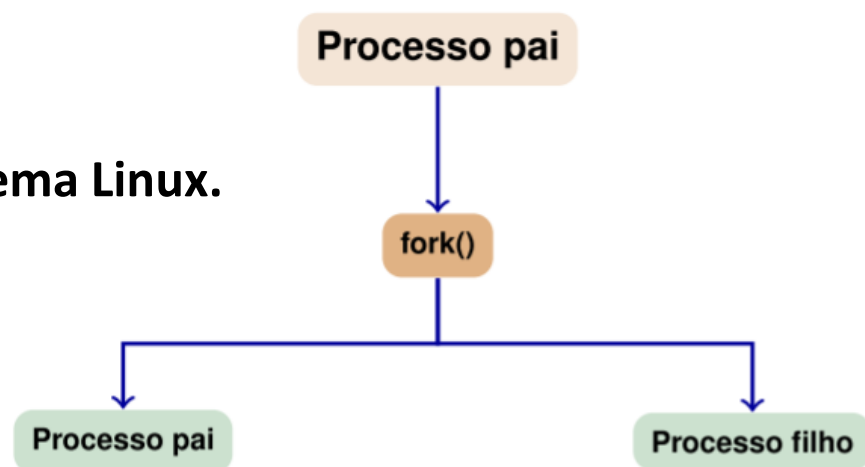
Hierarquia de Processos

- **Processos pai e filho**
 - Seja um processo A que cria um processo B
 - O processo A é chamado processo pai do processo B
 - O processo B é chamado processo filho do processo A

Dica:

a) Utilize o comando “ps xa” do sistema Linux.

b) Agora utilize o comando “ps fxa”.



Implementação de Processos

- Exemplo de THREADS
- Exemplo de FORK